Research Software Packaging for FAIR and Reproducible Analysis in EUCAIM

Version v.1.2

WP6 - July 2025

Table of Contents

Disclaimer

This document represents the first version of the user guide and is subject to change and evolution as the project progresses. As we gather more information and feedback, updates and new versions of this guide are expected to be released in the near future. Users are advised to refer to the latest version of the user guide for the most current information and instructions.

1. Introduction

This guide aims to help developers package research software as Docker containers, ensuring that the resulting containers are FAIR (Findable, Accessible, Interoperable, Reusable) and support reproducible and secure analysis. By following these steps, *Software Providers* can create and distribute Docker containers that are easy to use, maintain, and share within the EUCAIM Federated Processing Network.

EUCAIM analysis platforms

EUCAIM supports several computing models that define where and how the packed software is going to be deployed. The suitability of the model depends on the restrictions applicable on the level of data mobility, compute or hardware resources, and the permitted analysis tools.

		Analysis Platforms		
Compute Locations		JOBMAN	VIP	FEM
EUCAIM	Reference Node	X	-	-
EUCAIM	Federated Data Nodes	-	-	X
3rd party Infra.	EGI, European Grid Infrastructure	-	X	-

The analysis platforms are the frameworks responsible for orchestrating, managing, distributing (if applicable) and preparing the compute processing environment for the packed software. EUCAIM proposed the following analysis tools:

- JobMan is a Kubernetes container orchestrator that automates job creation, execution and scaling, with proper monitoring and logging and a possible CI/CD integration. It allows users to specify the resources needed for a job from the available flavors that have been defined (e.g. different memory, cpu and gpu usage), by passing them as a command line argument when submitting a job.

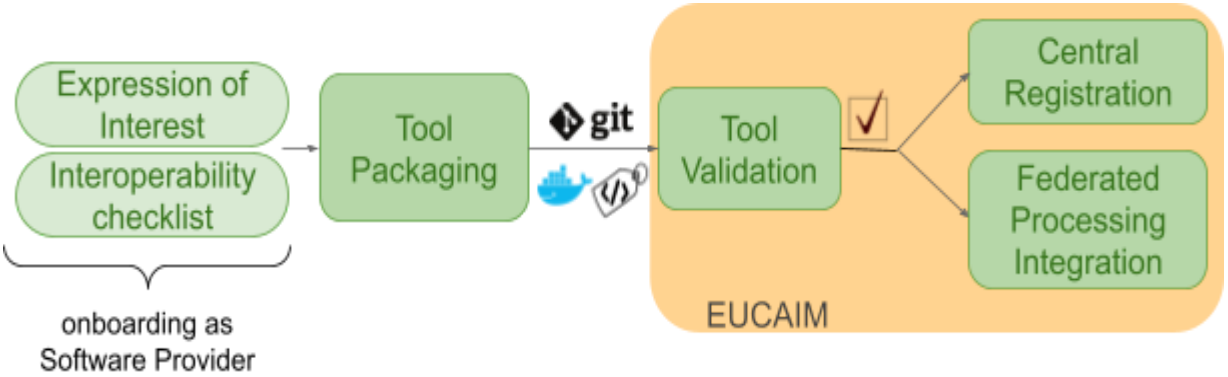
This platform has been tested in the UPV Reference Node. In this repository there is information about its installation, configuration, resources, as well as examples of the workflow and basic commands.

- VIP is a web portal for the simulation and processing of medical images. It leverages resources available in the biomed Virtual Organisation of the EGI e-infrastructure to offer an open service to academic researchers worldwide. VIP relies on:
  - Dirac for workload and data management on EGI compute and storage resources.
  - Boutiques for the application description, import and execution.

- FedManager is the federated processing orchestrator designed and developed at EUCAIM. It manage the distributed processing of EUCAIM Tools across the federated data nodes (FDN) in a secure manner, while delegating the actual execution to pluggable local engines (*i.e.* Docker engine). More information available at the following document: Architecture of the EUCAIM Federated Repository v3.

EUCAIM tool submission overview

A simplified overview of the protocol to integrate a new Tool at EUCAIM is described below:



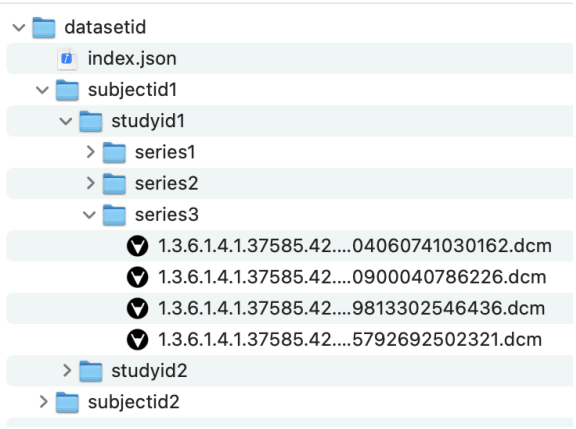
Where:

- *Tool:* either an analysis tool (e.g. radiomics extractor), a data preprocessing tool, or an AI model integrated in one of the federated learning frameworks (e.g. FedBiomed)
- *Onboarding as Software Provider*: protocol described as part of D2.2
- *Tool Validation*: first draft of protocol being generated here by WP5 and WP6.
- *Central Registration*: central registration and permissioning of the new Tool as a new EUCAIM resource

- *Federated Processing Integration:* integration test of the new Tool to the relevant EUCAIM analysis platforms

The applications can assume that the materialisator component will make the data available to the processing component in the format specified in EUCAIM's Common Data Model (CDM). The guidelines for organizing the imaging files are the following:

- Use a different directory per study, using the study id as the name of the folder. Use separate subfolders for each series too, using the series id or a meaningful name for each subfolder.
- It is optional, but recommended, to store all the directories with the studies of the same subject in a separate directory, using the id of the subject as the name of the directory.
- Include an index.json file in the root directory of the dataset with the structure detailed in the next figure. The hashed subject name is necessary to match with the clinical data. The schema is available in https://github.com/EUCAIM/upv-node-workstation-images/blob/main/index.schema.json




```
index.json:
[{'studyId': 'studyid1',
  'subjectName': 'hashed_subject_name',
  'series': [{'folderName':'series1'},
             {'folderName':'series2'},
             {'folderName':'series3'},
  'path': 'subjectid1/studyid1'},
  {'studyId': 'studyid2', ... }
]
```

Structure of the imaging files after materialisation. File structure (left) and content of the index.json file (right).

2. Requirements

Tool Requirements

Before containerizing the software, the following technical requirements for the application or tool willing to be packed should be considered:

- The tool should offer a command-line interface for **batch-mode** execution (non-interactive).
- The tool should be **stateless** between executions

- The tool should allow **rootless** executions under a **predefined user** (UID) and group (GID).
- Tool may be expected to operate under **restricted network** settings at runtime depending on the Federated Data Node (FDN). An example of these restrictions could be:
    - outgoing external TCP/IP connections (whitelisted)
    - no incoming or local TCP/IP connections
- The tool should support configuration via **command-line parameters**
- The tool should have clear and **configurable locations of inputs and outputs** via command line, being these files or directories. Note: it must be taken into account that the user will have read-only permissions on the input paths.
- The tool should **log** its output to standard output (stdout) and standard error (stderr)
- The tool should handle **termination signals** (like SIGTERM) gracefully, allowing it to clean up resources and shut down properly

Software Requirements

The system where the tool is being packed should fulfill a list of requirements:

- Operating System: Linux, macOS, or Windows with WSL2
- Docker
- Git

Additionally, consider that Software Providers should have access to:

- Docker registry
- CI/CD (Git) runner

3. Preparing Git Project

The first step is to create a structured Git repository for the project using the following template:

EUCAIM Tool Repository Template
https://github.com/EUCAIM/software-template-repository.git

The project files' structure and names can be freely adapted and extended from the following recommended structure:

```
my-analysis-tool-template/
├── Dockerfile
├── .hadolint.yml
├── .gitlab-ci.yml
├── README.md
├── LICENSE.md
├── src/
│   └── [..]
└── tests/
│   └── [..]
└── metadata/
    └── descriptor.yml
```

Note: It is recommended to create the software repository at the EUCAIM's Git community

4. Writing a Dockerfile

A Dockerfile is a script that contains a series of instructions on how to build a Docker image. It must be included in the Git project as described in the previous section.

Basic structure example

```
# Use an official base image relevant to your software (e.g., Python, R,
Ubuntu)

FROM python:3.9-slim

# Maintainer information
LABEL maintainer="your.email@example.com"
LABEL version="1.0"
LABEL description="A Python tool for data analysis."
LABEL image.source="https://github.com/username/repo"
LABEL image.revision="commit-sha"
LABEL image.version="tag-or-version"

# Set environment variables
ENV PYTHONUNBUFFERED=1

# Set defaults for build arguments
ARG USER_UID=2323
ARG USER_GID=2323

# Create non-root user and group, and prevent login
RUN groupadd -g $USER_GID eucaim && \
        useradd -r -u $USER_GID -g eucaim -d /home/eucaim -m -s
/usr/sbin/nologin eucaim

# Install any necessary dependencies of your tool
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Set working directory (placeholder, overridden at runtime)
WORKDIR /app

# Copy the tool code into the container
COPY . /app

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Set any necessary executable permissions for non-root user
RUN chown -R root:root /app && chmod 755 /app/your_script.py

####### Switch to non-root user  ############
USER eucaim

# Define command to run the tool (overridden at runtime)
CMD ["python", "/app/your_script.py"]
```

Although the previous Dockerfile is only an example, the following points when preparing your tool should be considered:

1. **Avoid Running as Root:**

- Running the container as a non-root user minimizes the potential impact of security vulnerabilities. The addgroup, adduser, and USER instructions ensure that the application runs with limited privileges.

2. **Use minimal and trusted Base Image:**
   - Use a minimal base image to reduce the attack surface and the size of the final image.
   - Use trusted providers to ensure the integrity of base images. Examples: Docker official images
3. **Dependency Management:**
   - Use package managers for handling dependencies (e.g., pip for Python, conda for Anaconda environments).
   - Use secure and up-to-date versions of dependencies, and specify exact versions where possible to avoid supply chain attacks.
4. **Minimize the installation of Packages:**
   - Avoid installing unnecessary packages and clean up package lists to reduce the potential for vulnerabilities and keep the image lightweight. Example: With apt, apt-get install -y --no-install-recommends install only the required packages and rm -rf /var/lib/apt/lists/* cleans up package lists.
5. **Make executables owned by root and not writable**
   - Ensure every executable in a container is owned by the root user, even if it is executed by the non-root user, and should not be world-writable
6. **Secure Permissions:**
   - Change the ownership of the application directory to the non-root user with chown -R appuser:appgroup /app
7. **Permit WORKDIR being overwritten**:
   - Please note that some platforms may override the WORKDIR at runtime. VIP for example will enforce a specific work directory (and entrypoint/script to execute) with a command of the following kind:

```
docker run
       --entrypoint=/bin/sh --rm
       -v /path/to/working/directory:/path/to/working/directory
       -w /path/to/working/directory
       dockerImage:v1.0.0 wrapper-script.sh
```

8. For more documentation about the handling of output directory refer to the particular platform specifications[1].
9. **Include HEALTHCHECK, specially for long batch runs:**
   - Using HEALTHCHECK permits defining a command that tests whether the container is still functioning correctly. The following example checks if the main script is still running by ensuring the last update for a specific log file is recent:

[1] e.g. in the case of the UPV reference node: https://github.com/chaimeleon-eu/workstation-images?tab=readme-ov-file#directories-for-mounting-volumes

```
# Dockerfile
[...]

CMD ["python", "/app/your_script.py"]

HEALTHCHECK    --interval=60s    --timeout=10s    --retries=3    CMD    python
/app/healthcheck.py || exit 1
```

```
# healthcheck.py

import os sys

def is_healthy():
    try:
        log_file = "/usr/src/app/log.txt"
        if os.path.exists(log_file):
            # Check file modified last 5 mins
          if (os.path.getmtime(log_file) > (time.time()-300)):
                return True
        return False
    except Exception as e:
        return False

if __name__ == "__main__":
    if is_healthy():
        sys.exit(0)
    else:
        sys.exit(1)
```

Specific instructions and best practices

The above Dockerfile is only an example that must be adapted in each case taking into account basic aspects about the management of base images, users, dependencies and packages, among others, to ensure the quality and security of the images built. The points to consider in each Dockerfile instruction are specified below:

FROM

When choosing an image, you must ensure it's built from a trusted source and keep it as small as possible.

- Use current **official images** as the basis for your images. Possible resources:
    - Docker Official Images are some of the most secure and dependable images on Docker Hub.
    - Verified Publisher images are high-quality images published and maintained by the organizations partnering with Docker, with Docker verifying the authenticity of the content in their repositories.
    - Docker-Sponsored Open Source are published and maintained by open source projects sponsored by Docker through an open source program.

- Specify base image **versions** (not the latest) to be able to rebuild your image under the same conditions and to have a control when the version needs to be updated.
- Choose a minimal base image that matches your requirements, for example the **slim** ones. It not only offers portability and fast downloads, but also shrinks the size of your image and minimizes the number of vulnerabilities introduced through the dependencies.
- Use **multi-stage builds** (advanced). You can use multiple FROM statements in your Dockerfile, specifying in each instruction a different base that begins a new stage of the build. You can selectively copy artifacts from one stage to another, leaving behind everything you don't want in the final image. More information: https://docs.docker.com/build/building/multi-stage/

LABEL

You must use labels to add some metadata to the images, in order to facilitate its organization within EUCAIM, record licensing information and to aid in automation.

- If you want to specify the **name** and **version** of the image that will appear in the EUCAIM images repository, you can set the following labels:

  ```
  LABEL name="tool"
  LABEL version="1.0"
  ```

- If your code repository is of type "Private" or it has not a license that allows redistribution, you need to include an **authorization** as a label:

  ```
  LABEL authorization="This Dockerfile is intended to build a container image that will be publicly accessible in the EUCAIM images repository."
  ```

- If you want to be contacted if any problems arise during the use of the image, you must add your **email** address in the label:

  ```
  LABEL maintainer="your.email@example.com"
  ```

- It is compulsory to add the following related labels so that the container registry is able to **link the image to the source Git code:**

  ```
  LABEL image.source="https://github.com/username/repo"
  LABEL image.revision="commit-sha"
  LABEL image.version="tag-or-version"
  ```

Note: Strings with spaces must be quoted (with double quotes) or the spaces must be escaped.

RUN

In this step, the necessary dependencies and packages must be installed and added to the base image for proper operation. To make the Dockerfile more readable, understandable and maintainable, you must consider:

- Chain related commands that can be run on one line using the **&&** operator.
- Separate long statements on **multiple lines** with backslashes and sort arguments alphabetically.
- Use **package managers** for handling dependencies (e.g., pip for Python, conda for Anaconda environments).
- Create a **requirements.txt** file to list all the dependencies or libraries you want to install. Use secure and up-to-date versions of dependencies, and specify exact versions where possible to avoid supply chain attacks. Thus, you will need a single RUN pip install in the Dockerfile. Note, you must use the -no-cache-dir tag when installing requirements.
- Minimize the installation of Packages. Avoid installing unnecessary packages and **clean up** package lists to reduce the potential for vulnerabilities and keep the image lightweight. Example:
    - With apt, apt-get install -y --no-install-recommends install only the required packages and rm -rf /var/lib/apt/lists/* cleans up package lists.

USER

Running the container as a **non-root** user minimizes the potential impact of security vulnerabilities. So, for all steps that can be run without privileges, the USER instruction must be used.

The USER instruction sets the username (or UID) and optionally the user group (or GID) to use as the default user and group for the remainder of the current stage. The specified user can be used for RUN instructions and at runtime, running the relevant ENTRYPOINT and CMD commands.

- Start by creating the user and group in the Dockerfile (consider an explicit UID/GID)
- Change the ownership of the application directory to the non-root user with chown -R appuser:appgroup /app.
- Use the instruction USER appuser before running commands as a non-root user.

WORKDIR

For clarity and reliability, you must always use absolute paths for your WORKDIR, taking into account that it can be overridden at runtime and that outputs are expected to be created there.

COPY

In case you use a relative path or point directly to the current directory (COPY .), you have to make sure that you are on the right path when creating the image.

CMD

The purpose of a CMD is to provide defaults for an executing container. These defaults can include an executable, or they can omit the executable, in which case you must specify an ENTRYPOINT instruction as well.

- There can only be **one CMD** instruction in a Dockerfile. If you list more than one CMD, only the last one takes effect.
- If you would like your container to run the same executable every time, then you should consider using **ENTRYPOINT** in combination with CMD.
- If the user specifies arguments to docker run then they will override the default specified in CMD and/or ENTRYPOINT.

ENV

Currently, no mandatory env variables

5. Building testing and sharing the container (using a CI/CD pipeline)

EUCAIM proposes the use of a CI/CD pipeline to ensure best practices when linting the Dockerfile, building, testing and validating the container image, and eventually pushing and sharing it through the EUCAIM docker container registry.

Note: When the Git source code of the Tool is at EUCAIM Git community, a local Gitlab docker runner is enabled.

Although any CI/CD engine could be used, the following snippet exemplifies the pipeline for a Gitlab CI/CD server. Software Tool providers should adapt it to their convenience, particularly:

- the *test* stage
- the managements of the secrets

Prepare Credentials

Store these credentials securely in GitLab > CI/CD variables.

- Obtain our Harbor server credentials and store these credentials securely in GitLab CI/CD variables:
  REGISTRY_USER,REGISTRY_PASSWORD
- Generate and store Docker Content Trust (DCT) keys to sign the new container
  - Manually generate signing keys using Docker CLI
    docker trust key generate <key_name>
  - Export the signing key to a file and encode it in base64

base64 ~/.docker/trust/private/<key_name> > signer.key.base64

- Secure storage in GitLab CI/CD variables.
  SIGNING_KEY

Prepare CI/CD pipeline

Here is a sample .gitlab-ci.yml file for a setup linting, building, testing, signing and pushing the container image

```
#.gitlab-ci.yml

image: docker:latest

services:
  - docker:dind

variables:
  DOCKER_DRIVER: overlay2
  REGISTRY: harbor.example.com
  REGISTRY_PROJECT: myproject
  REGISTRY_USER: $REGISTRY_USER
  REGISTRY_PASSWORD: $REGISTRY_PASSWORD
  DOCKER_CONTENT_TRUST: "1"
stages:
-      lint
-      project_structure
-      build
-      test
-      scan
-      validate_metadata
-      push

before_script:
    -  docker  login  -u  "$CI_REGISTRY_USER"  -p  "$CI_REGISTRY_PASSWORD"
"$CI_REGISTRY"
  - docker login -u "$REGISTRY_USER" -p "$REGISTRY_PASSWORD" "$REGISTRY"

lint:
  stage: lint
  script:
    - docker run --rm -i -v $CI_PROJECT_DIR/.hadolint.yml:/.hadolint.yaml
hadolint/hadolint < $CI_PROJECT_DIR/Dockerfile
  only:
    - /^release.*$/

project_structure:
  stage: project_structure
  script:
    - if [ ! -d "$CI_PROJECT_DIR/metadata" ]; then echo "Error: metadata
directory not found"; exit 1; fi
```

```
    - if [ ! -f "$CI_PROJECT_DIR/metadata/cwltool.yml" ]; then echo "Error:
metadata/cwltool.yml not found"; exit 1; fi
  only:
    - /^release.*$/

build:
  stage: build
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
                  -     docker     tag     $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
$REGISTRY/$REGISTRY_PROJECT/$CI_PROJECT_NAME:$CI_COMMIT_SHA
  only:
    - /^release.*$/


test:
  stage: test
  script:
    - docker run --rm $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA pytest tests
  only:
    - /^release.*$/

scan:
  stage: scan
  script:
       -   docker   run   --rm   -v   /var/run/docker.sock:/var/run/docker.sock
aquasec/trivy     image     --exit-code     1     --severity     HIGH,CRITICAL
$REGISTRY/$REGISTRY_PROJECT/$CI_PROJECT_NAME:$CI_COMMIT_SHA
  only:
    - /^release.*$/

sign:
  stage: sign
  script:
               -     echo     "$SIGNING_KEY"     |     base64     -d     >
~/.docker/trust/private/<key_filename>
    - docker trust signer add --key ~/.docker/trust/private/<key_filename>
signer $REGISTRY/$REGISTRY_PROJECT/$CI_PROJECT_NAME:$CI_COMMIT_SHA
                                -          docker          trust          sign
$REGISTRY/$REGISTRY_PROJECT/$CI_PROJECT_NAME:$CI_COMMIT_SHA

validate_metadata:
  stage: validate_metadata
  script:
    - docker run --rm -v $CI_PROJECT_DIR/metadata/cwltool.yml:/cwltool.yml
commonworkflowlanguage/cwltool --validate /cwltool.yml
  only:
    - /^release.*$/

push:
```

```
stage: push
script:
 - docker push $REGISTRY/$REGISTRY_PROJECT/$CI_PROJECT_NAME:$CI_COMMIT_SHA
only:
 - /^release.*$/
dependencies:
 - scan
```

.hadolint.yaml

6. Writing the command-line descriptor file

To facilitate the sharing and execution of command-line tools across different platforms, Software tool Providers are required to describe the inputs, outputs and execution environment of their command-line tool in a standardized way, making it easier to share and reuse tools in various computational environments.

Option 1: Boutiques

VIP relies on Boutiques for the command-line description. Boutiques is a system allowing to describe, publish, integrate and execute command-line applications across platforms. It relies on Linux containers to facilitate the application installation and sharing, and it uses a versatile JSON format to describe the command-line template, inputs and outputs. Below is an example of a simple descriptor, more complex examples are available here.

```
  {
"command-line": "echo '[STRING_INPUT]' [FILE_INPUT] &> [LOG]",
"container-image": {
"image": "ubuntu:latest",
"type": "docker"
},
"description": "This property describes the tool or application",
"inputs": [
{
       "default-value": "aStringValue",
       "description": "Describe the use and meaning of the parameter",
       "id": "str_input",
       "list": true,
       "max-list-entries": 3,
       "min-list-entries": 1,
```

```
            "name": "A string input",
            "type": "String",
            "value-key": "[STRING_INPUT]"
      },
      {
            "default-value": "/path/to/a/FileValue.txt",
            "description": "Describe the use and meaning of the parameter",
            "id": "file_input",
            "name": "A file input",
            "type": "File",
            "value-key": "[FILE_INPUT]"
      }
      ],
      "invocation-schema": {
      "$schema": "http://json-schema.org/draft-04/schema#",
      "additionalProperties": false,
      "dependencies": {},
      "description": "Invocation schema for example boutiques tool.",
      "properties": {
            "file_input": {
            "type": "string"
            },
            "str_input": {
            "items": {
                  "type": "string"
            },
            "maxItems": 3,
            "minItems": 1,
            "type": "array"
            }
      },
      "required": [
            "str_input",
            "file_input"
      ],
      "title": "Example Boutiques Tool.invocationSchema",
      "type": "object"
      },
      "name": "Example Boutiques Tool",
      "output-files": [
      {
            "description": "The output log file from the example tool",
            "id": "logfile",
            "name": "Log file",
            "path-template": "./test_temp/log-[FILE_INPUT].txt",
            "path-template-stripped-extensions": [
            ".txt",
            ".csv"
            ],
            "value-key": "[LOG]"
```

```
        }
        ],
        "schema-version": "0.5",
        "shell": "/bin/bash",
        "tool-version": "0.0.1"
}
```

Option 2: CWL

The Common Workflow Language (CWL) is widely used in the bioinformatics and scientific computing communities due to its ability to describe tools and workflows in a portable and scalable way.

CommandLineTool example:

```
cwlVersion: v1.2
class: CommandLineTool
baseCommand: ["python3", "/usr/local/bin/image_segmentation.py"]
hints:
  DockerRequirement:
       dockerPull: image_segmentation:release1.0

inputs:
  input_directory:
       type: Directory
       inputBinding:
         position: 1
         prefix: --data
       doc: "Input directory containing compressed NIFTI images (.nii.gz)"
       secondaryFiles: []
       extensionFields:
         data_type: "bioimage" # free list of key values

  output_directory:
       type: Directory
       inputBinding:
         position: 2
         prefix: --output_dir
       doc: "Output directory to store the segmented images"
       secondaryFiles: []
       extensionFields:
         data_type: "annotated_bioimage"
  model:
     type: string
     inputBinding:
       position: 3
```

```
      prefix: --model
    doc: "Model to use for segmentation"
    secondaryFiles: []
    extensionFields:
      data_type: "bioimage"
    enum: ["model-1", "model-2", "model-3"]
outputs:
  output_directory:
      type: Directory
      outputBinding:
      glob: $(inputs.output_directory.path)
      doc: "Output directory containing the list of segmented images"
      secondaryFiles: []
      extensionFields:
      data_type: "bioimage"

requirements:
  InlineJavascriptRequirement: {}
  InitialWorkDirRequirement:
      listing:
      - class: Directory
      location: $(inputs.input_directory.path)
      listing: $(inputs.input_directory.listing)
  ShellCommandRequirement: {}

arguments: []

stdout: $(inputs.input_directory.basename)-segmentation.log
```

7. Recommendation ensuring FAIR Principles

Findable

- **Metadata:** Ensure your Docker image has proper metadata (e.g., author, version, description).
- **Naming:** Use clear and descriptive names for your Docker images and tags.
- **Repository:** Publish your image in a public Docker registry (e.g., Docker Hub) with a descriptive README.

Accessible

- **Repository Access:** Ensure your Docker images are publicly accessible in a Docker registry.

- **Documentation:** Provide detailed documentation on how to pull and run the Docker container.
- **Licensing:** Include a clear and open license for your software.

Interoperable

- **Standards:** Maximize the use of widely-accepted standards and formats (e.g., JSON, CSV)
- **APIs:** Ensure your software exposes APIs that follow standard conventions.
- **Compatibility:** Make sure the container can run in various environments and is compatible with other tools in the analysis platform.

Reusable

- **Versioning:** Use version control for both your software and Docker images.
- **Documentation:** Provide extensive documentation, including examples and use cases.
- **Modularity:** Ensure your software is modular and can be easily integrated with other tools.

8. Provide Documentation

Create a comprehensive README file including usage examples, input/output specifications, and contact information:

1. Specify hardware requirements
    - CPU or GPU (Intel, NVIDIA, AMD, among others)
    - Memory
    - Storage
2. Specify network requirements
    - list outgoing connections (origin/destination; port)
    - list ingoing connections (origin/destination; port)
3. Include usage examples, input/output specifications, and contact information

Hardware Requirements

- **Processor:** Modern x86_64 or ARM64 CPU
- **Memory:** Minimum 2 GB RAM (4 GB recommended)
- **Storage:** Sufficient disk space for Docker images and data (Minimum 10 GB)

9. Final Considerations

- **Security:** Regularly update your base image and dependencies to include security patches.
- **Automation:** Use CI/CD pipelines to automate the building, testing, and deployment of Docker images.
- **Community:** Engage with the user community to gather feedback and improve the software.

By following this guide, you can ensure that your research software is packaged in a Docker container that is FAIR and reproducible, making it a valuable tool for the analysis platform and its users.